Web Development 2022-23 CSC7062 PROJECT STACKS OF WAX

by Gowthami Kumaresan (Student No: 40379728)

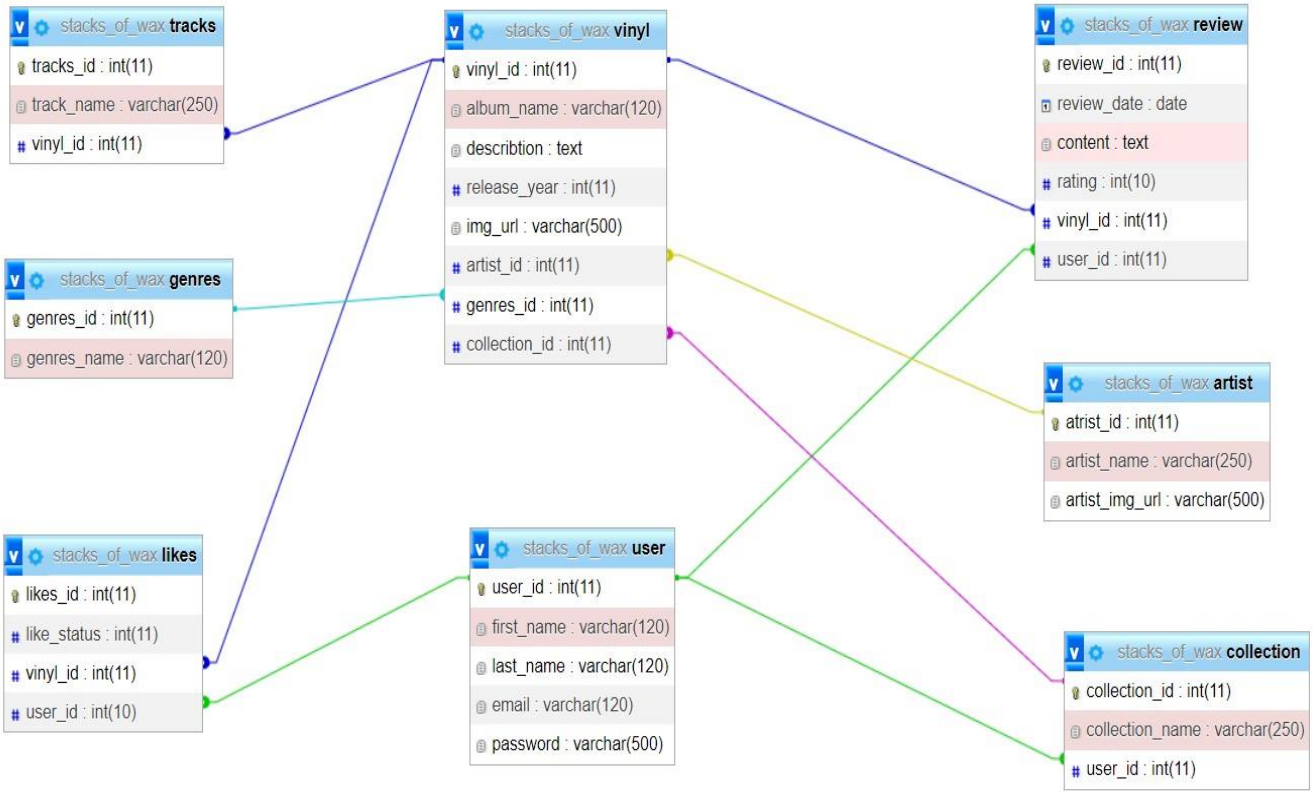
Video Demo: https://youtu.be/XnkPHvY0j-o

1. Introduction:

This document explains the design and development of the website "Stacks of Wax". It will enable users to browse vinyl music collections shared by other users. The platform will allow anonymous users to search and filter collections, while registered users will have access to additional features like adding, managing collections and sharing their collections with the community.

2. Entity Relationship (ER) diagram of the database:

The diagram below shows key entities such as user, collection, vinyl, and other entities used to implement data storage for the website.



This design ensures that data integrity and consistency are maintained with the appropriate number of entities. Entities are designed in such a way that all the relevant attributes and relationships were captured along with consideration of keys, normalisation, and constraints. The 'vinyl' entity has one-to-many relationships with entities 'tracks', 'likes', and 'review'. Moreover, 'vinyl' has many-to-one relationships with entities 'genres', 'artist', and 'collection'. In addition, the 'user' entity has one-to-many relationships with entities 'collection', 'likes', and 'review'. From the security viewpoint, the sensitive attribute 'password' in the 'user' entity is encrypted at the application level itself so that the database stores only encrypted data, thus ensuring user's data is protected at the storage level.

3. High-level description of the developed system:

The website's functionalities were designed and implemented in adherence to the specified system requirements. These requirements include a responsive, modern user interface (UI) design, browsing and searching of vinyl data on the system, filtering of vinyl collections based on criteria such as music genre, top-rated/most liked, and artist/band; User registration, and benefits for registered users such as formulating a catalogue of music records/vinyl, managing music record collections, liking and rating other members' vinyl collections, and commenting on members' vinyl collections. Furthermore, the website shall grant anonymous web users limited user rights and privileges.

3.1. User Interface and Database:

Plan was to start with UI design, Database design, and then proceed with back-end development for the system. UI design is built with the Bootstrap framework, offering benefits such as a responsive and intuitive user interface and cross-browser compatibility. For the database, MySQL is used which provided a scalable and robust solution for storing and managing large amounts of data related to vinyl records and user accounts.

3.2. Website walkthrough:

Homepage of the website features 'Most Liked Records' and Recently Added' sections which can be explored by both registered and anonymous users. A common header and footer are available for all the webpages except the Login and Registration webpages. Pagination[10] support in the website helps the user to navigate the webpage contents easily, thus improving user experience. The website's Navigation bar offers multiple menus, namely DISCOVER, GENRES, ARTISTS, & COLLECTIONS, to facilitate the searching and filtering of vinyl collections.

Search option available in the Navigation bar allows a user to explore vinyl by searching with any criteria such as Artist, Genre, Collection name, or Album name.

Registered users are provided with the facility to view their dashboards which show their total collections and total music records available in their profile. The user can view two charts: A pie chart that shows Genres in the user's collections, and a bar chart that shows the number of music records in the user's every individual collection.

Furthermore, users can view their individual collections through 'Library' section. Users can also create new collections and add new music records into collections through 'Create new collection' and 'Add Record' webpages respectively. 'Settings' webpage will allow the user to manage (edit or delete) the collections and music records from their profile.

Users can edit (including changing of password) or delete their profile info with 'My Profile' section.

4. Brief analysis of the development approach:

The "Stacks of Wax" website provides a range of features and functionality designed to enhance the user experience and enable efficient management of vinyl collections.

4.1. Usage of Nodejs and its modules:

Node.js is used as JS runtime environment, providing an efficient platform for running the server-side code and handling user requests. Core & third-party modules in Node.js are used to provide a range of features such as routing, middleware, and error handling.

The Node.js express framework, which is an optimized JavaScript library, is used to provide a lightweight and flexible framework for building the system. The express framework is based on the middleware module 'connect', which leverages the HTTP module. With the express module, routes are used to handle user navigation to various URIs.

To have the server respond with HTML content, a templating language was needed that can handle dynamic content injection from the REST[1] (Representational state transfer) API. EJS (Embedded JavaScript) is used for this purpose, as it helps to avoid manual HTML string construction when dealing with dynamic content. Nodejs express-session and cookie-parser are used for user login and to ensure the user session's temporary storage and security, keeping user data secure and protected from malicious attacks.

Middleware is a key concept of Express. Some of the middleware used in the website are routing middleware (to execute when a specific URL matches), Express static middleware (to support static files like style sheets or images), and express.urlencoded() middleware function (to parse the incoming request bodies in a middleware before the handlers).

4.2. API:

The Application Programming Interface (API) serves as an intermediary between the client and the database, facilitating CRUD functionality such as creating, reading, updating, and deleting data. This website employs an API that is built on top of the database, with a focus on CRUD functionality.

To adhere to the architectural constraints of a REST API, the API utilizes the commonly recognized JSON format for exchanging data with clients. The client code will use an HTTP request to ask/send the API data. The API will respond by either granting or denying the request. To test the API manually rather than code, the "Postman" app (Appendix [3]) was used which is a scalable API testing application.

The Axios library is used for server-side HTTP calls to the REST API, ensuring that data is transmitted quickly and efficiently between the server and client. Finally, the MySQL library is used to connect to the database with API, providing a fast and reliable way of retrieving and updating data stored in the database.

4.3. UI design:

Bootstrap, Custom CSS sheets, and EJS are instrumental in contributing to the front-end of the website. The Bootstrap[11,12] framework is employed for styling webpages. However, specific components are customized using CSS sheets (e.g., index.css for ensuring consistent and uniform styling of elements throughout the website, and register.css for the Login/Register page) based on the design requirements.

These customized styling sheets are utilized for styling fonts, buttons, as well as intuitive error and success messages. The website's design incorporates a logo that adheres to the 'stacks of wax' theme. The Appendix [1] provides a glimpse of the UI design of the homepage.

4.4. Search and Filter Functionality:

To elaborate on the search functionality, when a user enters a search term in the Search option, the term is inserted into a query string, and a GET request is initiated from the client-side. Axios endpoints facilitate the transmission of the query string to the server-side API. On the server-side, the API issues a database read request to MySQL using the search-term in the SQL query. The resulting data is converted into JSON format and returned to the client-side for rendering.

It is worth noting that the above-mentioned search mechanism applies to all search options available on the website.

4.5. User registration and login Functionalities:

Let's explore the development of the user registration functionality for the system. The user will be prompted to enter their first and last name, email address, and password. To ensure the security of the system, users are required to create a strong password that meets the following criteria: a minimum of 8 characters, at least 1 uppercase letter, at least one lowercase letter, at least one number, and at least one special character. This is handled through the use of a regular expression.

When the user registration is attempted, client-side code is utilized to verify whether user data already exists by sending a request to the API, which then reads the database. Rendering of any error or success message on the website is handled with the 'connect-flash'[3,4] dependency, which is based on the Nodejs express-session module. An error message is displayed if the user already exists. If the user data is not already present, a new user is registered in the database, and the login[13] page is presented to the user. To ensure password security, the 'bcrypt'[5] dependency is utilized to encrypt the password before it is forwarded to the database for storage.

When a user enters their login credentials on the login page, client-side code will retrieve the user's details with the assistance of the API, which reads data from the database. If the user information does not exist, the user will be informed of an error and redirected to the Registration page. If the user information does exist, the password will be decrypted utilizing the 'bcrypt' dependency and compared to the password that was entered by the user. Once a successful login has been verified, the homepage will be displayed to the user.

4.6. Registered user's functionality:

A registered user has the ability to add and manage new collections and music records.

Image upload functionality is available for edit and add record webpages. Uploading of image to the website uses three key dependencies: path (for directory navigation), fs (for file operations), and multer (for saving the image). The multer[6] dependency uploads the image to the server's storage directory, and the server-side sends a write request for the image name to the database via the API endpoint.

For the Tracklists in 'Add new Record' webpage, the client-side code organizes them into an array, converts the array to JSON format, and sends them to the server-side via the API endpoint. The JSON data is then iterated through in a loop on the server-side, and each track is added to the database.

In the 'Add new Record' webpage, promises[9] are used to handle asynchronous operations ensuring that database insertions are complete before returning a response. The code (Appendix [2]) uses promises to wait for the insertion of a new artist before using their ID in the next operation. Similarly, promises are used to wait for the insertion of a vinyl record before adding the tracks associated with that vinyl. Each SQL query for inserting an artist or track is wrapped in a promise that either resolves with data or rejects with an error. Promises allow for better error handling and ensure that data is inserted in the correct order.

In the website, registered user can manage their collections and records with edit and delete options in the 'Settings' webpage which is implemented with POST and DELETE methods for the form using 'method-override'[7] dependency and updated in DB via API endpoints. Same flow is also applicable for edit and delete operations in 'My Profile' webpage. A registered user can give Likes, reviews, and ratings to the music albums while anonymous users can only see reviews and ratings of registered users. POST method is used to save the Likes, reviews, and ratings[2] into the database.

User's dashboard webpage provides the facility to view the collections and records info in the form of pie chart and bar graph. To get the information needed for the chart and graph, a request was made to an API endpoint, with a URL parameter of the user's id being passed. The library used to achieve these chart and graph is Google chart[8].

4.7. Security and authentication:

The website's security and authentication were strengthened by leveraging Nodejs express-session and cookie-parser dependencies and using the session object variable to store user information that could be utilized across multiple pages. During the login process, user credentials were authenticated against the stored user and password data in the database. Upon finding a match, a session object variable was set and used to verify the login status on subsequent pages. This process was very useful to prevent unauthorized access to secure pages via direct URL entry, which would trigger a redirect to the home page. These measures were adopted to ensure the website's security and privacy and provide an optimized user experience.

4.8. Implementation best practices:

Implementation for the website is carried out with a focus on key characteristics such as modularization[14], maintenance, code efficiency etc. For example, folders such as 'styles', 'js_functions', and 'middleware' are common for the whole website. In addition, same reusable code for common design elements such as headers, and footers were used throughout the website. This approach was very useful in minimizing code repetition, simplifying the codebase and making it easier to maintain. Implementation of routes for each individual webpage served well in maintenance and future enhancements. As a result, updates and modifications can be implemented efficiently and consistently by locating the specific source files.

5. Instructions to deploy and run the system:

Prerequisites: XAMPP, Nodejs, and VS Code installed on the system.

- Extract the Zip file and copy the folders 'source', and 'database' to DESKTOP.
- Open PhpMyAdmin MySQL. Create a new database '40379728'. Import the DB file '40379728.sql' from the folder 'database'. DB file will be imported successfully.
- Open two VS Code application instances: Open the 'API' folder from the 'source' folder in one instance and open the 'stacks_of_wax' folder from the 'source' folder in another instance.
- Run the following command in the terminal of both application instances to install the express web framework.
    - **npm install express --save**
- Run the following commands in the terminal of 'stacks_of_wax' folder instance:
    - **npm install ejs --save**
    - **npm install express-session --save**
    - **npm install cookie-parser --save**
    - **npm install express axios --save**
    - **npm install connect-flash --save**
    - **npm install multer --save**
    - **npm install method-override --save**
    - **npm install bcrypt --save**
- Run the following commands in the terminal of 'API' folder instance:
    - **npm install mysql --save**
    - **npm install dotenv --save**
- Run the following commands in the terminal of both application instances:
    - **npm install nodemon --save-dev**
- Run the following commands in the terminal of both application instances to run the website:
    - **npm start**
- Launch the website with the command http://localhost:3000/ in the browser.
- To test the registered user's functionalities, use either Register or the following account details:
    - **Account1** email: user1@gmail.com , password: user1@WD123
    - **Account2** email: kelly@gmail.com , password: kelly@WD123

6. References:

1. Build REST API with Node js & Express https://www.youtube.com/watch?v=pKd0Rpw7O48
2. Build A Star Rating System https://medium.com/geekculture/how-to-build-a-simple-star-rating-system-abcbb5117365
3. Display Flash Messages using Node JS Connect-flash Module https://www.youtube.com/watch?v=GYm2ooHTgSg
4. Flash Messages In Node.Js Using The Connect-Flash Package https://www.workfall.com/learning/blog/how-to-show-flash-messages-in-node-js-using-the-connect-flash-package/

5. Hashing passwords using Bcrypt with Nodejs https://medium.com/@manishsundriyal/a-quick-way-for-hashing-passwords-using-bcrypt-with-nodejs-8464f9785b67
6. File Uploads in Node.js using Multer https://www.youtube.com/watch?v=EVOFt8Its6I
7. method-override dependency https://expressjs.com/en/resources/middleware/method-override.html
8. Google Chart https://www.w3schools.com/ai/ai_google_chart.asp
9. JS Promise object https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
10. Pagination in Rest API https://dev.to/yemiklein/how-to-implement-pagination-in-rest-api-5deg
11. Bootstrap 5 Grids https://www.w3schools.com/bootstrap5/bootstrap_grid_basic.php
12. slideshow - Carousel https://getbootstrap.com/docs/5.0/components/carousel/
13. Login form https://mdbootstrap.com/docs/standard/extended/login/
14. Project structure for an Express REST API https://www.coreycleary.me/project-structure-for-an-express-rest-api-when-there-is-no-standard-way

7. Appendix

[1] Sample view of Home page

[2] JS 'Promises' code to add music record into the database from server-side

```
  if (new_artist) {
    let createsql = `INSERT INTO artist (atrist_id,artist_name, artist_img_url) VALUES (NULL,?,?);`;
    const values = [new_artist, artist_img];
    connection.query(createsql, values, (err, data, fields) => {
      if (err) {
        res.json({ err });
        throw err;
      } else {
        // get the artist_id of the newly created artist
        artist_id = data.insertId;
        // execute the update query
        executeUpdateQuery(artist_id);
      }
    });
  } else {
    executeUpdateQuery(artist_id);
  }
  function executeUpdateQuery(artist_id) {
    let createsql = `INSERT INTO vinyl (vinyl_id, album_name, describtion, release_year, img_url, artist_id,
genres_id, collection_id) VALUES (NULL, ?, ?, ?, ?, ?, ?, ?);`;
    const values = [
      album_name,
```

```
    album_name,
    describtion,
    release_year,
    vinyl_img,
    artist_id,
    genres_id,
    collection_id,
  ];
  // execute insert query and wait for the result
  const insertVinylPromise = new Promise((resolve, reject) => {
    connection.query(createsql, values, (err, data, fields) => {
      if (err) {
        reject(err);
        throw err;
      } else {
        resolve(data);
      }
    });
  });
  insertVinylPromise.then((data) => {
    let vinylid = data.insertId;
    //add track
    let promises = [];
```

```
  for (let i = 0; i < tracks_list.length; i++) {
    const trackName = tracks_list[i];
    let tracklist = `INSERT INTO tracks (tracks_id, track_name, vinyl_id) VALUES (NULL, ?, ?);`;
    const values = [trackName, vinylid];
    let promise = new Promise((resolve, reject) => {
      connection.query(tracklist, values, (err, data, fields) => {
        if (err) {
          reject(err);
        } else {
          resolve(data);
        }
      });
    });
    promises.push(promise);
  }
  Promise.all(promises)
    .then((results) => {
      let respObj = {
        id: vinylid,
      };
      res.json({ respObj });
    })
```

[3] Sample Postman get results for music records based on genre with page 3 and total data 20

GET http://localhost:4000/genres/All/3/20 Send

Params Authorization Headers (11) Body Pre-request Script Tests Settings Cookies

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 39 ms Size: 10.71 KB Save as Example

Pretty Raw Preview Visualize JSON


```
{
  "data": [
    {
      "vinyl_id": 4,
      "album_name": "Rumours",
      "describtion": "Rumours is the eleventh studio album by British-American rock band Fleetwood Mac, February–August 1976 and released February 4, 1977.",
      "release_year": 1977,
      "img_url": "https://i.discogs.com/WpuYpr4IADSy49WTIw4r6YW5p_AWCNQVZg6XvaQ0Yw8/rs:fit/g:sm/q:90/h:583/w:600/czM6Ly9kaXNjb2dz/LWRhdGFiYXNlLWlt/YWdlcy9SLTM3NDg4MS0xMTkzMDMzNzU0/LmpwZWc.jpeg",
      "artist_id": 2,
      "genres_id": 10,
      "collection_id": 16
    },
    {
      "vinyl_id": 5,
      "album_name": "Born To Run",
      "describtion": "Born to Run is the third studio album by American singer-songwriter Bruce Springsteen, released on August 25, 1975, by Columbia Records.",
```